

MODEL CONTEXT PROTOCOL

MCP 핵심 정리

공식 문서 기준으로 정리한 개념·구조·활용·보안

출처: modelcontextprotocol.io/docs/getting-started/intro

왜 MCP가 필요한가?

- 지금까지: LLM에 프롬프트로 질문하거나, 파일·이미지를 직접 업로드하는 것이 전부였다.
- 한계: 매번 사람이 자료를 넣어줘야 하고, 외부 도구·데이터와는 연결되지 않는다.
- 질문: AI가 내 캘린더·Notion·DB에 직접 연결될 수는 없을까?

→ 이 연결을 표준화한 것이 MCP.

정의

MCP란 무엇인가?

- MCP(Model Context Protocol) = AI 애플리케이션을 외부 시스템에 연결하는 오픈소스 표준.
- 연결 대상: 데이터 소스(파일·DB) · 도구(검색·계산) · 워크플로(특화 프롬프트).
- 비유: "AI 애플리케이션을 위한 USB-C 포트" — 표준 하나로 어디에나 연결한다.

이름 뜯어보기

Model · Context · Protocol

Model	AI 언어 모델(LLM) — 예: Claude, GPT, Gemini
Context	모델이 작업에 필요한 배경정보·환경 (데이터, 도구, 작업 상태, 리소스)
Protocol	표준화된 통신 규칙과 절차

왜 표준이 필요했나 — $N \times M$ 문제

- 표준 전: M 개 LLM $\times$ N 개 도구 = $M \times N$ 개의 개별 연동을 각각 구현해야 했다.
- 서비스마다 도구 스키마·함수를 직접 작성·유지 $\rightarrow$ 유지보수 부담이 폭발.
- MCP 이후: 한 번 만들면 어디서나 재사용 ($N+M$).

"build once, integrate everywhere" — 한 번 만들면 어디서나 통합.

MCP의 등장과 확산

2024.11	Anthropic이 MCP를 오픈소스로 공개 (개발자 커뮤니티의 실제 니즈 반영)
2025	OpenAI · Google · Microsoft 등 주요 AI 기업이 잇따라 채택
현재	Claude · ChatGPT · VS Code · Cursor 등 폭넓게 지원

→ 특정 회사의 기술이 아니라 업계 공통 표준으로 자리잡음.

무엇이 달라졌나

MCP 이전 vs 이후

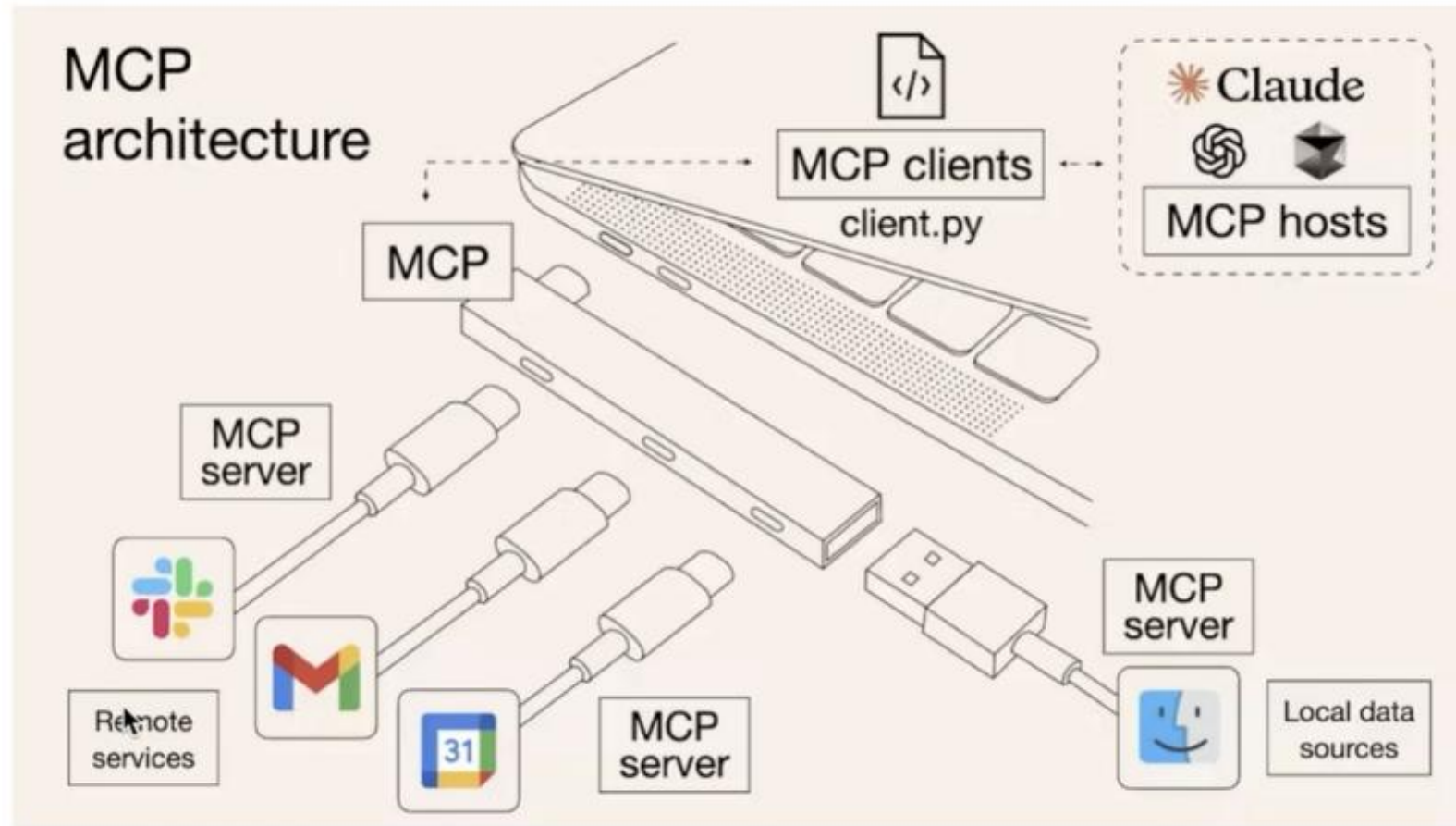
MCP 이전 (복잡)	MCP 이후 (단순)
① DB 연결 함수 작성 ② 쿼리 생성 함수 작성 ③ 결과 포맷 함수 작성 ④ 에러 핸들링 구현 ⑤ AI 도구별로 각각 설정	MCP 서버 연결 한 번으로 끝 "지난 주 가장 많이 팔린 상품 10개 보여줘"

MCP의 3요소

MCP Host	사용자와 LLM을 담은 앱 본체 (지휘관)
MCP Client	Host 안에서 서버와 1:1로 통신하는 중개자
MCP Server	도구·데이터를 제공하는 쪽

관계: Host ⊃ Client →(통신)→ Server

MCP 아키텍처 — 공식 다이어그램



Host(Claude 등) 안의 Client가, USB-C 허브처럼 여러 MCP Server(Slack·Gmail·로컬 등)에 표준 하나로 연결된다.

MCP Host — 지휘관

- 모든 것을 결정하는 앱 본체. LLM을 품고 있다.
- 사용자 요청 이해·분석 → 필요한 도구 결정 → 작업 계획 수립 → 최종 응답 생성.
- 예: Claude Desktop, Cursor.

② MCP CLIENT

MCP Client — 중개자

- Host와 Server 사이를 잇는 커넥터.
- 각 MCP 서버와 1:1 연결을 유지한다.
- Host 애플리케이션 안에서 실행 — 별도 앱이 아니라 Host의 부품.
- 도구를 직접 실행하지 않고 통신만 중개한다.

MCP Server — 실제 도구 제공

외부 데이터·도구를 MCP 표준에 맞게 노출한다. 예: GitHub MCP, Notion MCP

서버가 제공하는 3대 프리미티브

Tools	실행 (도구)	API 호출·데이터 변환 — 모델(LLM)이 호출
Resources	읽기 (자료)	파일·DB·웹 내용 — 앱이 가져와 컨텍스트로
Prompts	템플릿	미리 정의된 명령·워크플로 — 사용자가 호출

MCP 전송 방식 — 2가지

stdio	표준 입출력. 같은 PC의 로컬 서버와 1:1 직접 통신, 가장 단순. 예) Claude Desktop이 로컬 파일 시스템에 접근
Streamable HTTP	원격 서버와 HTTP로 통신. 필요 시 SSE로 응답을 스트리밍. 예) 인터넷 너머의 원격 MCP 서버

참고: 예전의 'HTTP+SSE 단독' 방식은 구버전(deprecated)으로, 지금은 Streamable HTTP로 통합됨. WebSocket은 공식 표준 전송 방식이 아니다.

REST API와 MCP — 같은 점 / 다른 점

MCP 원격 통신은 HTTP 위에서 동작한다. 하지만 REST API는 아니다 — 메시지 규약은 JSON-RPC.

	REST API	MCP
운반 (전송)	HTTP	HTTP(원격) · stdio(로컬)
메시지 방식	URL + GET/POST/PUT/DELETE	단일 엔드포인트 + JSON-RPC method
예시	GET /todos/5	{ "method": "tools/call" }
인증	Bearer · OAuth2	Bearer · OAuth2 (동일)

→ HTTP 토대(전송·인증)는 공유하되, 메시지 규약은 REST가 아니라 JSON-RPC. (비유: 도로=HTTP · REST=택시 · MCP=버스)

MCP로 할 수 있는 일 vs 없는 일

- MCP는 만능이 아니다.
- 하지만 적절한 영역에서는 정말 효율적이다.
- 한계를 아는 것이 성공적 활용의 시작이다.

가능 vs 불가능

무엇이 되고, 무엇이 안 되나

할 수 있는 일	할 수 없는 일
· 데이터 접근·조회 (정형 API, 읽기 중심) · 자동화·워크플로 (모니터링·알림) · 정보 통합·분석 (다중 소스·크로스 플랫폼)	· 실시간 스트리밍 (밀리초 응답) · 상태 의존적 긴 작업 (트랜잭션) · 고성능 대용량 계산 (네트워크 오버헤드)

이유: MCP는 요청-응답 패턴 + 네트워크를 거치는 구조이기 때문.

적합한 일 / 부적합한 일

적합 	월간 보고서 자동화 — GitHub·Jira·Slack·Sheets MCP 코드 리뷰 자동화 — GitHub·File System·Playwright MCP
부적합 ✕	실시간 주식 거래 — 지연·규제·책임 소재 문제 대용량 비디오 처리 — 메모리·대역폭 한계

MCP 사용 시 보안 위협

- 민감한 데이터 노출 — 고객 정보·내부 문서·운영 로그 유출 가능성.
- 무단 명령 실행 — 인증되지 않은 사용자가 임의 명령을 실행하는 취약점.
- API 키·인증 토큰 탈취 — 키 유출로 비용 청구 또는 데이터 유실.
- 운영 시스템 침해 — 백도어 설치·리소스 오용으로 시스템 전체 위협.

키·토큰 관리

- API 키·비밀키는 코드에 직접 쓰지 않고 .env 파일에 분리한다.
- .env는 .gitignore에 넣어 Git 등 버전 관리에 올라가지 않게 한다.
- 환경 변수 로딩 실패 시 안전한 기본값 제공 또는 예외 처리로 서비스 중단을 막는다.

★ 우리 실습에서도 `cli_project`의 .env를 .gitignore로 제외했다.

최소 권한 원칙

- 최소 권한: 파일시스템=읽기전용+특정 디렉토리, DB=SELECT 전용 계정, API=읽기 키 우선.
- 실행 명령 필터링: 허용 명령만 화이트리스트로 등록, 입력값 검증(SQL·웹 인젝션 방지).
- 접근 제어: 사용자 유형별 기능 제한, JWT·OAuth2·세션 인증.
- 인증 실패 처리: 일반화된 오류 메시지, 반복 실패 시 잠금·차단, 모든 실패 로그 기록.

정리

한 장 요약

- MCP = AI를 외부 도구·데이터에 잇는 개방형 표준 ("AI의 USB-C").
- 3요소: Host(지휘관) · Client(중개자) · Server(도구 제공).
- 전송 방식: stdio(로컬) · Streamable HTTP(원격).
- 서버 기능: Tools · Resources · Prompts.
- 만능은 아님 — 요청/응답에 맞는 영역에서 강력하며, 키·권한 등 보안이 필수.